ECALL/OCALL 메커니즘의 코드 레벨 구현 분석

개요

이 문서는 Intel SGX SDK의 ECALL/OCALL 메커니즘을 실제 소스 코드 흐름을 따라가면서 상세히 분석한 결과입니다. Edger8r이 생성한 프록시 코드부터 TRTS의 디스패치 로직, 어셈블리 레벨의 컨텍스트 스위칭까지 전체 코드 경로를 추적하여 분석했습니다.

1. Edger8r 코드 생성 과정

1.1 EDL 파일 파싱 (Lexer.mll, Parser.mly, Ast.ml)

Edger8r는 OCaml로 구현된 코드 생성 도구로, EDL(Enclave Definition Language) 파일을 파싱하여 프록시 코드를 생성합니다.

핵심 데이터 구조 (Ast.ml):

```
type enclave_content = {
file_shortnm : string;
enclave_name : string;
include_list : string list;
comp_defs : composite_type list;
tfunc_decls : trusted_func list;
ufunc_decls : untrusted_func list;
}
```

```
type trusted_func = {
tf_fdecl : func_decl;
tf_is_priv : bool;
tf_is_switchless : bool;
}

type untrusted_func = {
uf_fdecl : func_decl;
uf_fattr : func_attr;
uf_allow_list : string list;
uf_propagate_errno : bool;
uf_is_switchless : bool;
}
```

파싱 과정:

1. `Lexer.mll` 에서 EDL 파일을 토큰으로 분해
2. `Parser.mly` 에서 문법 분석을 통해 AST(Abstract Syntax Tree) 생성
3. `parse_enclave_ast` 함수에서 AST를 `enclave_content` 구조체로 변환

1.2 마샬링 구조체 생성 (CodeGen.ml)

ECALL 마샬링 구조체 생성:

```
let gen_ecall_marshal_struct (tf: Ast.trusted_func) =
gen_marshal_struct tf.Ast.tf_fdecl "" true
```

```
let mk_ms_member_decl (pt: Ast.parameter_type) (declr: Ast.declarator) (isecall:
let qual = if is_const_ptr pt then "const " else "" in
let aty = Ast.get_param_atype pt in
let tystr = Ast.get_tystr aty in
let ptr = if is_foreign_array pt then "* " else "" in
let field = mk_ms_member_name declr.Ast.identifier in
(* 문자열 길이 변수 생성 *)
let str_len = if need_str_len_var pt then sprintf "\tsize_t %s_len;\n" field els
in
let dmstr = get_array_dims declr.Ast.array_dims in
sprintf "\t%s%s%s %s%s;\n%s" qual tystr ptr field dmstr str_len
```

생성되는 마샬링 구조체 예시:

```
// int foo(char* buf, size_t len) ECALL의 경우
typedef struct ms_foo_t {
char* ms_buf; // 포인터 매개변수
size_t ms_buf_len; // 문자열 길이 (자동 생성)
size_t ms_len; // 크기 매개변수
int ms_retval; // 반환값
} ms_foo_t;
```

1.3 프록시 함수 생성

Trusted 프록시 함수 생성:

```
let gen_tproxy_proto (fd: Ast.func_decl) =
let parm_list = (* 매개변수 리스트 생성 *) in
let retval_parm_str = gen_parm_retval fd.Ast.rtype in
if fd.Ast.plist = [] && fd.Ast.rtype = Ast.Void then
sprintf "sgx_status_t SGX_CDECL %s(void)" fd.Ast.fname
else if fd.Ast.plist = [] then
sprintf "sgx_status_t SGX_CDECL %s(%s)" fd.Ast.fname retval_parm_str
else if fd.Ast.rtype = Ast.Void then
sprintf "sgx_status_t SGX_CDECL %s(%s)" fd.Ast.fname parm_list
else
sprintf "sgx_status_t SGX_CDECL %s(%s, %s)" fd.Ast.fname retval_parm_str parm_li
```

Untrusted 프록시 함수 생성:

```
let gen_uproxy_com_proto (fd: Ast.func_decl) (prefix: string) =
let retval_parm_str = gen_parm_retval fd.Ast.rtype in
let eid_parm_str =
if fd.Ast.rtype = Ast.Void then sprintf "(sgx_enclave_id_t %s" eid_name
else sprintf "(sgx_enclave_id_t %s, " eid_name in
let parm_list =
List.fold_left (fun acc pd -> acc ^ ", " ^ gen_parm_str pd)
```

```
retval_parm_str fd.Ast.plist in
let fname =
if !g_use_prefix then sprintf "%s_%s" prefix fd.Ast.fname
else fd.Ast.fname
in "sgx_status_t " ^ fname ^ eid_parm_str ^ parm_list ^ ")"
```

2. ECALL 코드 흐름 분석

2.1 URTS (Untrusted Runtime System) - ECALL 진입점

sgx_ecall 함수 (routine.cpp):

```
extern "C"
sgx_status_t sgx_ecall(const sgx_enclave_id_t enclave_id, const int proc, const
{
return _sgx_ecall(enclave_id, proc, ocall_table, ms, false);
}

static
sgx_status_t _sgx_ecall(const sgx_enclave_id_t enclave_id, const int proc, const
{
if (proc < 0)
{
return SGX_ERROR_INVALID_FUNCTION;
```

```
}

CEnclave* enclave = CEnclavePool::instance()->ref_enclave(enclave_id);

if(!enclave)
return SGX_ERROR_INVALID_ENCLAVE_ID;

sgx_status_t result = SGX_ERROR_UNEXPECTED;
{
result = enclave->ecall(proc, ocall_table, ms, is_switchless);
}
{
CEnclavePool::instance()->unref_enclave(enclave);
}

return result;
}
```

CEnclave::ecall 함수 (enclave.cpp):

```
sgx_status_t CEnclave::ecall(const int proc, const void *ocall_table, void *ms,
{
if(se_try_rdlock(&m_rwlock))
```

```
{
if(m_destroyed)
{
se_rdunlock(&m_rwlock);
return SGX_ERROR_ENCLAVE_LOST;
}

// Switchless ECALL 처리
if (m_switchless && is_switchless)
{
int need_fallback = 0;
sgx_status_t ret = SGX_ERROR_UNEXPECTED;

ret = g_sl_funcs.sl_ecall_func_ptr(m_switchless, proc, ms, &need_fallback);

if (likely(!need_fallback))
{
se_rdunlock(&m_rwlock);
return ret;
}
}

// 일반 ECALL 또는 fallback된 switchless ECALL 처리
```

```
CTrustThread *trust_thread = get_tcs(proc);

unsigned ret = SGX_ERROR_OUT_OF_TCS;



if(NULL != trust_thread)

{

if (NULL == m_ocall_table)

{

m_ocall_table = (sgx_ocall_table_t *)ocall_table;

}



ret = do_ecall(proc, m_ocall_table, ms, trust_thread);

if(SGX_PTHREAD_EXIT == ret)

trust_thread->reset_ref();

else

trust_thread->decrease_ref();

}



if(AbnormalTermination() || ret != SE_ERROR_READ_LOCK_FAIL)

{

se_rdunlock(&m_rwlock);

}

return error_trts2urts(ret);

}
```

```
else
{
return SGX_ERROR_ENCLAVE_LOST;
}
}
```

2.2 TRTS (Trusted Runtime System) - ECALL 처리

do_ecall 함수 (trts_ecall.cpp):

```
sgx_status_t do_ecall(int index, void *ms, void *tcs)
{
sgx_status_t status = SGX_ERROR_UNEXPECTED;
if(ENCLAVE_INIT_DONE != get_enclave_state())
{
return status;
}
thread_data_t *thread_data = get_thread_data();
if( (NULL == thread_data) ||
((thread_data->stack_base_addr == thread_data->last_sp) &&
( (TCS_POLICY_BIND != g_global_data.thread_policy) ||
(SGX_PTHREAD_EXIT == _pthread_tls_get_state()) ||
(index == ECMD_ECALL_PTHREAD))))
{
```

```
status = do_init_thread(tcs, false);
if(0 != status)
{
return status;
}
}
thread_data = get_thread_data();
if(thread_data->stack_base_addr == thread_data->last_sp)
{
// Root ECALL 처리
sgx_wrpkru(0);


if(_pthread_enabled())
{
jmp_buf buf = {0};
if(0 == setjmp(buf))
{
_pthread_tls_store_context((void*)buf);
status = random_stack_advance<0x800>(trts_ecall, index, ms);
}
else
{
thread_data->last_sp = thread_data->stack_base_addr;
```

```
status = SGX_PTHREAD_EXIT;
}
if(ECMD_ECALL_PTHREAD == index || SGX_PTHREAD_EXIT == status)
{
_pthread_tls_store_state(SGX_PTHREAD_EXIT);
}
_pthread_tls_destructors();
_pthread_wakeup_join(ms);
}
else
{
status = random_stack_advance<0x800>(trts_ecall, index, ms);
}
}
else
{
status = trts_ecall(index, ms);
}


if ((TCS_POLICY_BIND != g_global_data.thread_policy) &&
(thread_data->stack_base_addr == thread_data->last_sp))
{
tc_set_idle();
```

```
}

return status;
}
```

trts_ecall 함수:

```
static sgx_status_t trts_ecall(uint32_t ordinal, void *ms)
{
sgx_status_t status = SGX_ERROR_UNEXPECTED;

if (unlikely(g_is_first_ecall))
{
thread_data_t *thread_data = get_thread_data();
if (thread_data->last_sp != thread_data->stack_base_addr)
{
return SGX_ERROR_ECALL_NOT_ALLOWED;
}

sgx_spin_lock(&g_ife_lock);
if (g_is_first_ecall)
{
// 전역 객체 초기화
```

```
init_global_object();

g_is_first_ecall = false;

}

sgx_spin_unlock(&g_ife_lock);

}



void *addr = NULL;

status = get_func_addr(ordinal, &addr);

if(status == SGX_SUCCESS)

{

ecall_func_t func = (ecall_func_t)addr;



sgx_lfence();



status = func(ms);

}

return status;

}
```

ECALL 함수 주소 획득:

```
static sgx_status_t get_func_addr(uint32_t ordinal, void **addr)

{

if(ordinal == (uint32_t)ECMD_ECALL_PTHREAD)
```

```
{
*addr = (void*) _pthread_thread_run;
return SGX_SUCCESS;
}

// 일반 사용자 정의 ECALL
sgx_status_t status = is_ecall_allowed(ordinal);
if(SGX_SUCCESS != status)
{
return status;
}

*addr = const_cast<void *>(g_ecall_table.ecall_table[ordinal].ecall_addr);
if(!sgx_is_within_enclave(*addr, 0))
{
return SGX_ERROR_UNEXPECTED;
}

return SGX_SUCCESS;
}
```

ECALL 권한 검사:

```
static sgx_status_t is_ecall_allowed(uint32_t ordinal)

{

if(ordinal >= g_ecall_table.nr_ecall)

{

return SGX_ERROR_INVALID_FUNCTION;

}

thread_data_t *thread_data = get_thread_data();

sgx_lfence();



if(thread_data->last_sp == thread_data->stack_base_addr)

{

// Root ECALL, 권한 비트 확인

if (g_ecall_table.ecall_table[ordinal].is_priv)

return SGX_ERROR_ECALL_NOT_ALLOWED;

return SGX_SUCCESS;

}

ocall_context_t *context = reinterpret_cast<ocall_context_t*>(thread_data->last_

if(context->ocall_flag != OCALL_FLAG)

{

abort();

}

uintptr_t ocall_index = context->ocall_index;
```

```
if(ocall_index >= g_dyn_entry_table.nr_ocall)

{

return SGX_ERROR_INVALID_FUNCTION;

}

return (g_dyn_entry_table.entry_table[ocall_index * g_ecall_table.nr_ecall + ord

}
```

3. OCALL 코드 흐름 분석

3.1 TRTS에서 OCALL 시작

sgx_ocall 함수 (trts_ocall.cpp):

```
sgx_status_t sgx_ocall(const unsigned int index, void *ms)

{

// OCALL 인덱스 유효성 검사

if((index != 0) && !is_builtin_ocall((int)index) &&

static_cast<size_t>(index) >= g_dyn_entry_table.nr_ocall)

{

return SGX_ERROR_INVALID_FUNCTION;

}



// sgx_ocall 실행

sgx_status_t status = do_ocall(index, ms);
```

```
return status;
}
```

do_ocall 함수 (__morestack):

```
#define do_ocall __morestack
extern "C" sgx_status_t __morestack(const unsigned int index, void *ms);
```

3.2 어셈블리 레벨 OCALL 처리 (trts_pic.S)

do_ocall 어셈블리 구현:

```
DECLARE_LOCAL_FUNC do_ocall
push %xbp
mov %xsp, %xbp

# 매개변수를 스택에 저장
mov %xdi, 2*SE_WORDSIZE(%xbp)
mov %xsi, 3*SE_WORDSIZE(%xbp)

# 확장 기능 레지스터 저장 및 정리
READ_TD_DATA xsave_size
sub %xax, %xsp # xregs 저장을 위한 버퍼 할당
mov $0x3f, %xax
```

```
not %xax

and %xax, %xsp # xsave는 64바이트 정렬 필요

mov %xsp, %xcx # xsave 포인터


sub $(20*SE_WORDSIZE), %xsp # GPR 및 기타 정보를 위한 20개 슬롯

mov %xcx, SE_WORDSIZE*19(%xsp) # xsave 주소


# 비휘발성 레지스터 저장 (xsp 제외)

mov %xbx, SE_WORDSIZE*14(%xsp)

mov %xsi, SE_WORDSIZE*13(%xsp)

mov %xdi, SE_WORDSIZE*12(%xsp)

mov %xbp, SE_WORDSIZE*11(%xsp)

mov %r12, SE_WORDSIZE*10(%rsp)

mov %r13, SE_WORDSIZE* 9(%rsp)

mov %r14, SE_WORDSIZE* 8(%rsp)

mov %r15, SE_WORDSIZE* 7(%rsp)


# 확장 기능 레지스터 저장 및 정리

mov SE_WORDSIZE*19(%xsp), %xdi # xsave 포인터

READ_TD_DATA xsave_size

mov %xax, %xcx

shr $2, %xcx # xsave 크기를 dword 단위로

xor %xax, %xax
```

```
cld

rep stos %eax, %es:(%xdi)


mov SE_WORDSIZE*19(%xsp), %xdi # xsave 포인터

mov %xdi, (%xsp)

call save_xregs

lea_pic SYNTHETIC_STATE, %xdi

mov %xdi, (%xsp)

call restore_xregs

lfence


# 입력 매개변수를 사용하여 xdi와 xsi 설정

mov SE_WORDSIZE*12(%xsp), %xdi

mov SE_WORDSIZE*13(%xsp), %xsi


# OCALL 인덱스를 스택에 저장

mov $OCALL_FLAG, %xax

mov %xax, SE_WORDSIZE*4(%xsp) # OCALL_FLAG 저장

mov %xdi, SE_WORDSIZE*5(%xsp) # ocall_index 저장


# 내부 스택 컨텍스트 저장

READ_TD_DATA self_addr

mov %xax, %xbx
```

```
# update_ocall_lastsp 호출
mov %xsp, %xdi
call update_ocall_lastsp # xax: td.last_sp


mov SE_WORDSIZE*12(%xsp), %xdi # xdi 복원
mov SE_WORDSIZE*13(%xsp), %xsi # xsi 복원
# 외부 스택 컨텍스트 복원
mov first_ssa_gpr(%xbx), %xdx
mov ssa_bp_u(%xdx), %xbp
mov ssa_sp_u(%xdx), %xsp
# EEXIT 레지스터 설정
mov -1*SE_WORDSIZE(%xax), %xbx # 반환 주소
mov $SE_EEXIT, %xax # EEXIT leaf


# 모든 GPR 정리 (xax, xbx, xdi, xsi 제외)
xor %xcx, %xcx
xor %xdx, %xdx
xor %r8, %r8
xor %r9, %r9
xor %r10, %r10
xor %r11, %r11
xor %r12, %r12
```

```
xor %r13, %r13

xor %r14, %r14

xor %r15, %r15


# 상태 플래그를 미리 정의된 값으로 설정

add %xdx, %xdx # OF = SF = AF = CF = 0; ZF = PF = 1

cld # DF = 0


ENCLU

END_FUNC
```

update_ocall_lastsp 함수:

```
extern "C"

uintptr_t update_ocall_lastsp(ocall_context_t* context)

{

thread_data_t* thread_data = get_thread_data();


uintptr_t last_sp = 0;


last_sp = thread_data->last_sp;


context->pre_last_sp = last_sp;
```

```
if (context->pre_last_sp == thread_data->stack_base_addr)
{
context->ocall_depth = 1;
} else {
ocall_context_t* context_pre = reinterpret_cast<ocall_context_t*>(context->pre_l
context->ocall_depth = context_pre->ocall_depth + 1;
}


thread_data->last_sp = reinterpret_cast<uintptr_t>(context);


return last_sp;
}
```

3.3 URTS에서 OCALL 처리

sgx_ocall 함수 (routine.cpp):

```
extern "C"
int sgx_ocall(const unsigned int proc, const sgx_ocall_table_t *ocall_table, voi
{
assert(trust_thread != NULL);
CEnclave* enclave = trust_thread->get_enclave();
assert(enclave != NULL);
return enclave->ocall(proc, ocall_table, ms);
```

```
}
```

CEnclave::ocall 함수:

```
int CEnclave::ocall(const unsigned int proc, const sgx_ocall_table_t *ocall_tabl
{
int error = SGX_ERROR_UNEXPECTED;

if (is_builtin_ocall(proc))
{
se_rdunlock(&m_rwlock);
if ((int)proc == EDMM_ALLOC)
error = ocall_emm_alloc(ms);
else if ((int)proc == EDMM_MODIFY)
error = ocall_emm_modify(ms);
}
else
{
// proc이 ocall_table 범위 내에 있는지 검증
if(NULL == ocall_table ||
(proc >= ocall_table->count))
{
return SGX_ERROR_INVALID_FUNCTION;
```

```
}

if (m_switchless)
{
g_sl_funcs.sl_ocall_fallback_func_ptr(m_switchless);
}

se_rdunlock(&m_rwlock);
bridge_fn_t bridge = reinterpret_cast<bridge_fn_t>(ocall_table->ocall[proc]);
error = do_ocall(bridge, ms);
}

if (!se_try_rdlock(&m_rwlock))
{
error = SE_ERROR_READ_LOCK_FAIL;
}
else if(m_destroyed)
{
se_rdunlock(&m_rwlock);
error = SE_ERROR_READ_LOCK_FAIL;
}
return error;
```

```
}
```

3.4 OCALL 복귀 처리

do_oret 함수:

```
sgx_status_t do_oret(void *ms)
{
thread_data_t *thread_data = get_thread_data();
uintptr_t last_sp = thread_data->last_sp;
ocall_context_t *context = reinterpret_cast<ocall_context_t*>(thread_data->last_
if(0 == last_sp || last_sp <= (uintptr_t)&context)
{
return SGX_ERROR_UNEXPECTED;
}
// 최소 1개의 ecall 프레임과 1개의 ocall 프레임이 스택에 있어야 함
if(last_sp > thread_data->stack_base_addr - 30 * sizeof(size_t))
{
return SGX_ERROR_UNEXPECTED;
}
if(context->ocall_flag != OCALL_FLAG)
{
return SGX_ERROR_UNEXPECTED;
}
```

```
if(context->pre_last_sp > thread_data->stack_base_addr
|| context->pre_last_sp <= (uintptr_t)context)
{
return SGX_ERROR_UNEXPECTED;
}

thread_data->last_sp = context->pre_last_sp;
asm_oret(last_sp, ms);
// 여기에 도달하면 안 됨
return SGX_ERROR_UNEXPECTED;
}
```

asm_oret 어셈블리 구현:

```
DECLARE_GLOBAL_FUNC asm_oret
mov %xsp, %xbx
mov %xdi, SE_WORDSIZE(%xsp)
mov %xsi, 2*SE_WORDSIZE(%xsp)
mov SE_WORDSIZE(%xbx), %xsp # thread_data.last_sp 복원

# 확장 기능 레지스터 복원
mov 19*SE_WORDSIZE(%xsp), %xdi
mov %xdi, (%xsp)
```

```
call restore_xregs

lfence



# memset_s

xor %xax, %xax

mov 11*SE_WORDSIZE(%xsp), %xcx

sub %xdi, %xcx

sub $SE_WORDSIZE, %xcx

shr $2, %xcx

cld

rep stos %eax,%es:(%xdi)



mov 2*SE_WORDSIZE(%xbx), %xax # ocall 반환값



# 레지스터 복원

mov 7*SE_WORDSIZE(%xsp), %r15

mov 8*SE_WORDSIZE(%xsp), %r14

mov 9*SE_WORDSIZE(%xsp), %r13

mov 10*SE_WORDSIZE(%xsp), %r12

mov 11*SE_WORDSIZE(%xsp), %xbp

mov 12*SE_WORDSIZE(%xsp), %xdi

mov 13*SE_WORDSIZE(%xsp), %xsi

mov 14*SE_WORDSIZE(%xsp), %xbx
```

```
mov %xbp, %xsp

pop %xbp



ret

# 여기에 도달하면 안 됨

ud2

END_FUNC
```

4. EENTER/EEXIT 명령어 처리

4.1 Enclave 진입점 (enclave_entry)

enclave_entry 어셈블리 구현 (trts_pic.S):

```
DECLARE_GLOBAL_FUNC enclave_entry

# 사용하지 않는 범용 레지스터 정리

xor %xdx, %xdx

add %xdx, %xdx # OF = SF = AF = CF = 0; ZF = PF = 1

cld # DF = 0

xor %r8, %r8

xor %r9, %r9

xor %r10, %r10

xor %r11, %r11
```

```
xor %r12, %r12

xor %r13, %r13

xor %r14, %r14

xor %r15, %r15



# 신뢰할 수 있는 스택으로 전환

cmp $0, %xax

jne .Ldo_handler # 예외 상태 처리

READ_TD_DATA last_sp

cmp $0, %xax

jne .Lswitch_stack

GET_STACK_BASE %xbx # last_sp == 0이면 sp를 스택 베이스로 설정

sub $STATIC_STACK_SIZE, %xax # 정적 스택을 위한 공간 확보

.Lswitch_stack:

xchg %xsp, %xax

push %xcx

push %xbp



.cfi_def_cfa_offset 2 * SE_WORDSIZE

.cfi_offset xbp, -2 * SE_WORDSIZE

mov %xsp, %xbp

.cfi_def_cfa_register xbp
```

```
CLEAN_XFLAGS


# 레지스터 저장
sub $(6*SE_WORDSIZE), %xsp
mov %xax, -1*SE_WORDSIZE(%xbp) # xsp_u
mov %xdx, -3*SE_WORDSIZE(%xbp) # cssa
mov %xbx, -4*SE_WORDSIZE(%xbp) # TCS
mov %xsi, -5*SE_WORDSIZE(%xbp) # XSI
mov %xdi, -6*SE_WORDSIZE(%xbp) # XDI


# 확장 기능 레지스터 정리
sub $(4*SE_WORDSIZE), %xsp


lea_pic SYNTHETIC_STATE, %xdi
mov %xdi, (%xsp)
call restore_xregs
lfence
add $(4*SE_WORDSIZE), %xsp


# C 코드로 전환
mov -6*SE_WORDSIZE(%xbp), %xdi # index
mov -5*SE_WORDSIZE(%xbp), %xsi # ms
mov -4*SE_WORDSIZE(%xbp), %xdx # TCS
```

```
mov -3*SE_WORDSIZE(%xbp), %xcx # cssa

call enter_enclave

mov %xax, %xbx



.Lexit_enclave:

# 확장 기능 레지스터 정리

lea_pic SYNTHETIC_STATE, %xdi

mov %xdi, (%xsp)

call restore_xregs

lfence



# AEX 알림 처리

mov -3*SE_WORDSIZE(%xbp), %xax # cssa

cmp $0, %xax

jne .Lrestore_untrust_stack



READ_TD_DATA self_addr # xax: td

cmp $0, %xax # td가 초기화되었는지 확인

je .Lrestore_untrust_stack

mov first_ssa_gpr(%xax), %xdx

movb aex_notify(%xdx), %cl

cmpb $0, %cl

je .Lrestore_untrust_stack
```

```
# aexnotify 비트를 일시적으로 비활성화하고

# td.aex_notify_flag를 설정하여 EEXIT 전에 aexnotify가 활성화되었음을 표시

movb $0, aex_notify(%xdx)

movl $0x1, aex_notify_flag(%xax)



.Lrestore_untrust_stack:

# xdi와 xsi 설정

mov $OCMD_ERET, %xdi

mov %xbx, %xsi



# 스택 복원

mov -1*SE_WORDSIZE(%xbp), %xdx # xdx: xsp_u

mov %xbp, %xsp

pop %xbp # xbp_u

pop %xbx # ret_u

mov %xdx, %xsp # xsp_u



.Lclear_and_exit_enclave:

# xax, xbx, xdi, xsi를 제외한 모든 GPR 정리

xor %xcx, %xcx

xor %xdx, %xdx

xor %r8, %r8

xor %r9, %r9
```

```
xor %r10, %r10

xor %r11, %r11

xor %r12, %r12

xor %r13, %r13

xor %r14, %r14

xor %r15, %r15



# 상태 플래그를 미리 정의된 값으로 설정

add %xdx, %xdx # OF = SF = AF = CF = 0; ZF = PF = 1

cld # DF = 0



# EEXIT

mov $SE_EEXIT, %xax # EEXIT leaf

ENCLU



# 여기에 도달하면 안 됨

ud2

END_FUNC
```

4.2 레지스터 상태 관리

EENTER 시 레지스터 상태:

- XAX: TCS.CSSA
- XBX: TCS 주소

- XCX: EENTER 다음 명령어 주소
- XDI: Enclave 진입 이유
- XSI: 마샬링 구조체 포인터

EEXIT 시 레지스터 상태:

- XAX: SE_EEXIT (leaf 값)
- XBX: 반환 주소
- XDI: 명령 코드 (OCMD_ERET 등)
- XSI: 반환 데이터

5. 매개변수 마샬링 과정

5.1 데이터 검증 및 복사 로직

sgx_is_within_enclave 함수 (trts.cpp):

```
int sgx_is_within_enclave(const void *addr, size_t size)
{
size_t start = reinterpret_cast<size_t>(addr);
size_t end = 0;
size_t enclave_start = (size_t)g_enclave_base;
size_t enclave_end = enclave_start + (size_t)g_enclave_size - 1;

if(size > 0)
{
end = start + size - 1;
```

```
}
else
{
end = start;
}
if( (start <= end) && (start >= enclave_start) && (end <= enclave_end) )
{
return 1;
}
return 0;
}
```

sgx_is_outside_enclave 함수:

```
int sgx_is_outside_enclave(const void *addr, size_t size)
{
size_t start = reinterpret_cast<size_t>(addr);
size_t end = 0;
size_t enclave_start = (size_t)g_enclave_base;
size_t enclave_end = enclave_start + (size_t)g_enclave_size - 1;

if(size > 0)
{
```

```
end = start + size - 1;

}

else

{

end = start;

}

if( (start <= end) && ((end < enclave_start) || (start > enclave_end)) )

{

return 1;

}

return 0;

}
```

5.2 OCALL 메모리 할당

sgx_ocalloc 함수:

```
void * sgx_ocalloc(size_t size)

{

// 현재 SSA에서 외부 스택 주소 읽기

thread_data_t *thread_data = get_thread_data();

ssa_gpr_t *ssa_gpr = reinterpret_cast<ssa_gpr_t *>(thread_data->first_ssa_gpr);

size_t addr = ssa_gpr->REG(sp_u);
```

```
// u_rsp가 신뢰할 수 없는 주소를 가리키는지 확인

if(!sgx_is_outside_enclave(reinterpret_cast<void *>(addr), sizeof(size_t)))

{

abort();

}



// 할당하기에는 크기가 너무 큼

if(addr < size)

{

abort();

}



// 할당된 메모리의 시작 주소 계산

addr -= size;

addr &= ~(static_cast<size_t>(OC_ROUND - 1)); // 스택 정렬을 위해



// 할당된 메모리가 enclave와 겹치면 enclave 중단

if(!sgx_is_outside_enclave(reinterpret_cast<void *>(addr), size))

{

abort();

}



// 외부 스택을 프로빙하여 스택 가드 페이지를 건너뛰지 않도록 보장
```

```
size_t first_page = TRIM_TO_PAGE(ssa_gpr->REG(sp_u) - 1);

size_t last_page = TRIM_TO_PAGE(addr);



// 무한 루프 방지

if (last_page == 0)

{

abort();

}



// 컴파일러 최적화를 피하기 위해 volatile 사용

for(volatile size_t page = first_page; page >= last_page; page -= SE_PAGE_SIZE)

{

// 페이지를 프로빙하기 전에 외부 스택 주소 업데이트

ssa_gpr->REG(sp_u) = page;



*reinterpret_cast<uint8_t *>(page) = 0;

}



// SSA의 외부 스택 주소를 할당된 주소로 업데이트

ssa_gpr->REG(sp_u) = addr;



return reinterpret_cast<void *>(addr);
```

```
}
```

sgx_ocfree 함수:

```
void sgx_ocfree()
{
// ECALL 스택 프레임
// last_sp -> | |
// -------------
// | ret_addr |
// | xbp_u |
// | xsp_u |

thread_data_t *thread_data = get_thread_data();
ssa_gpr_t *ssa_gpr = reinterpret_cast<ssa_gpr_t *>(thread_data->first_ssa_gpr);
uintptr_t *addr = reinterpret_cast<uintptr_t *>(thread_data->last_sp);
uintptr_t usp = *(addr - 3);
if(!sgx_is_outside_enclave(reinterpret_cast<void *>(usp), sizeof(uintptr_t)))
{
abort();
}
ssa_gpr->REG(sp_u) = usp;
}
```

6. 보안 메커니즘

6.1 스택 카나리 검사

check_static_stack_canary 함수:

```
extern uintptr_t __stack_chk_guard;
int check_static_stack_canary(void *tcs)
{
size_t *canary = TCS2CANARY(tcs);
if ( *canary != (size_t)__stack_chk_guard)
{
return -1;
}
return 0;
}
```

init_static_stack_canary 함수:

```
static void init_static_stack_canary(void *tcs)
{
size_t *canary = TCS2CANARY(tcs);
*canary = (size_t)__stack_chk_guard;
}
```

6.2 메모리 경계 검사

모든 포인터 매개변수는 `sgx_is_within_enclave()` 또는 `sgx_is_outside_enclave()` 함수를 통해 적절한 메모리 영역에 있는지 검증됩니다.

6.3 OCALL 컨텍스트 검증

```
if(context->ocall_flag != OCALL_FLAG)
{
// OCALL 프레임이 유효하지 않으면 enclave 중단
abort();
}
```

7. 성능 최적화

7.1 Switchless ECALL/OCALL

Switchless 메커니즘은 컨텍스트 스위칭 오버헤드를 줄이기 위해 도입되었습니다:

```
if (m_switchless && is_switchless)
{
int need_fallback = 0;
sgx_status_t ret = SGX_ERROR_UNEXPECTED;

ret = g_sl_funcs.sl_ecall_func_ptr(m_switchless, proc, ms, &need_fallback);
```

```
if (likely(!need_fallback))
{
se_rdunlock(&m_rwlock);
return ret;
}
}
```

7.2 스택 랜덤화

```
status = random_stack_advance<0x800>(trts_ecall, index, ms);
```

스택 포인터를 랜덤하게 조정하여 사이드 채널 공격을 완화합니다.

8. 에러 처리

8.1 에러 코드 변환

error_trts2urts 함수:

```
sgx_status_t CEnclave::error_trts2urts(unsigned int trts_error)
{
if(trts_error == (unsigned int)SE_ERROR_READ_LOCK_FAIL)
{
return SGX_ERROR_ENCLAVE_LOST;
}
```

```
// tRTS가 직접 외부 에러 코드를 반환할 수 있으므로 변환할 필요 없음
if(EXTERNAL_ERROR != (trts_error >> MAIN_MOD_SHIFT))
{
SE_TRACE(SE_TRACE_WARNING, "trts return error %x, it should be urts/trts bug\n",
return SGX_ERROR_UNEXPECTED;
}


return (sgx_status_t)trts_error;
}
```

결론

Intel SGX SDK의 ECALL/OCALL 메커니즘은 다음과 같은 복잡한 과정을 통해 구현됩니다:

1. **Edger8r 코드 생성**: EDL 파일을 파싱하여 마샬링 구조체와 프록시 함수 생성
2. **ECALL 처리**: URTS에서 시작하여 TRTS의 디스패치 로직을 통해 실제 함수 호출
3. **OCALL 처리**: TRTS에서 시작하여 어셈블리 레벨 컨텍스트 스위칭을 통해 URTS로 전환
4. **어셈블리 레벨 처리**: EENTER/EEXIT 명령어를 통한 실제 컨텍스트 스위칭
5. **보안 메커니즘**: 메모리 경계 검사, 스택 카나리, OCALL 컨텍스트 검증 등

이러한 메커니즘을 통해 SGX는 신뢰할 수 있는 영역과 신뢰할 수 없는 영역 간의 안전한 통신을 보장하면서도 성능을 최적화합니다.